

Towards Sustainable Fine-Tuning of Large Language Models: A Multi-Objective Benchmark of Parameter-Efficient Adaptation Methods

[Ashraful Islam Tanzil]
[United International University]
[ahmedtanzil174@gmail.com]

Abstract

Fine-tuning large language models (LLMs) is increasingly dominated by parameter-efficient fine-tuning (PEFT) methods such as LoRA, QLoRA, LoRA-FA, and LISA, which reduce trainable parameters and memory footprint relative to full fine-tuning. However, most published comparisons optimize a single axis — typically task accuracy or training speed — and rarely report energy consumption or carbon emissions with the same rigor. This paper proposes a multi-objective benchmarking framework that jointly evaluates PEFT methods along four dimensions: task performance, computational efficiency (time, FLOPs, peak memory), energy consumption (kWh), and estimated carbon footprint (kgCO₂eq), using region-specific grid carbon intensity. We outline a controlled experimental protocol across multiple open-weight LLM backbones (ranging from $\sim 1\text{B}$ to $\sim 13\text{B}$ parameters), a standardized set of downstream tasks, and a shared measurement stack (CodeCarbon / AIMeter-style tracking) to ensure comparability. Method trade-offs are analyzed using Pareto-frontier and weighted multi-criteria scoring rather than single-metric leaderboards, allowing practitioners to select adaptation strategies under explicit sustainability constraints. This document presents the motivation, related work, and a fully specified methodology; empirical results, statistical analysis, and discussion are left for the completed study.

Keywords: parameter-efficient fine-tuning, LoRA, QLoRA, sustainable AI, green machine learning, carbon footprint, multi-objective optimization, LLM efficiency

1 Introduction

Large language models (LLMs) have grown from millions to hundreds of billions of parameters, and adapting them to downstream tasks via full fine-tuning is increasingly impractical for most organizations in terms of GPU memory, training time, and cost. Parameter-efficient fine-tuning (PEFT) methods — including Low-Rank Adaptation (LoRA) (Hu et al., 2021), its quantized variant QLoRA (Dettmers et al., 2023), factorized variants such as LoRA-FA (Zhang et al., 2023), and layer-selective approaches such as LISA (Liu et al., 2024) — have emerged as the dominant response, updating a small fraction of parameters while approximating the performance of full fine-tuning.

At the same time, a growing body of work has drawn attention to the environmental cost of training and operating LLMs, quantifying energy use, water consumption, and carbon emissions across the model lifecycle (Jiang et al., 2024; Jegham et al., 2025; Dauner and Socher, 2025; Kaur et al., 2026). Despite this, sustainability metrics are rarely integrated into PEFT benchmarking: papers proposing new adaptation methods typically report accuracy, trainable-parameter counts, and sometimes wall-clock time, but seldom energy draw or carbon-equivalent emissions measured under comparable conditions. Where energy or efficiency is

measured (Chen et al., 2024; Husom et al., 2025; Iftikhar et al., 2025; Rithvik and Arya, 2025), studies typically focus on a single method, a single hardware setting, or a narrow task set, which limits cross-method comparability.

This paper addresses that gap by proposing a multi-objective benchmark that positions sustainability as a first-class evaluation criterion alongside accuracy and speed, following the spirit of recent multi-objective hyperparameter optimization work for LLM and RAG systems (Barker et al., 2025) and multi-objective fine-tuning for code generation (Morte et al., 2025).

1.1 Research Questions

- **RQ1:** How do popular PEFT methods (LoRA, QLoRA, LoRA-FA, LISA, and full fine-tuning as a baseline) compare when evaluated jointly on task performance, training time, peak memory, energy consumption, and estimated carbon emissions?
- **RQ2:** Which PEFT method(s) dominate the Pareto frontier when performance is traded off against energy/carbon cost, and how does this ranking change across model sizes (1B–13B) and task types (classification, summarization, instruction following)?
- **RQ3:** How sensitive are the sustainability metrics to hyperparameters intrinsic to each method (e.g., LoRA rank r , quantization bit-width, LISA layer-sampling frequency), and can a low-cost proxy (e.g., early-epoch energy trajectory) predict full-run carbon cost?
- **RQ4:** What is the practical carbon/energy saving of PEFT methods relative to full fine-tuning, expressed in absolute terms (kWh, kgCO₂eq) and normalized per performance point gained?

1.2 Contributions

- A reproducible, open-source multi-objective benchmarking protocol for PEFT methods that reports performance, efficiency, energy, and carbon metrics under a shared measurement stack.
- An empirical comparison (to be completed via simulation/experiments) of LoRA, QLoRA, LoRA-FA, LISA, and full fine-tuning across multiple backbones and tasks.
- A Pareto-frontier and weighted multi-criteria analysis framework that lets practitioners choose adaptation strategies under explicit accuracy–sustainability trade-off preferences.
- A discussion of measurement pitfalls (hardware variance, regional grid carbon intensity, idle-power accounting) relevant to reproducible green-AI benchmarking.

2 Related Work

2.1 Parameter-Efficient Fine-Tuning Methods

LoRA (Hu et al., 2021) freezes the pretrained weight matrices and injects trainable low-rank decomposition matrices into each layer, dramatically reducing the number of trainable parameters while preserving inference-time architecture. QLoRA (Dettmers et al., 2023) extends this by fine-tuning over a 4-bit quantized frozen backbone, further reducing memory footprint and enabling fine-tuning of larger models on commodity GPUs. LoRA-FA (Zhang et al., 2023) freezes one of the two low-rank projection matrices to reduce activation memory and communication overhead during fine-tuning. LISA (Liu et al., 2024) takes a different, non-low-rank approach, selectively unfreezing a small, randomly sampled subset of transformer layers per step, and reports memory savings competitive with LoRA. Broader surveys (Xu et al., 2023;

Mao et al., 2024) catalogue the wider PEFT landscape (adapters, prompt/prefix tuning, BitFit, and hybrid methods), situating the low-rank family used in this study within that space.

2.2 Efficiency and Energy Measurement of LLM Training and Fine-Tuning

A parallel line of work quantifies the computational and environmental cost of LLM training and adaptation. Chen et al. (2024) propose SpeedUp, MemoryUp, and EnergyUp as multi-dimensional metrics for characterizing fine-tuning efficiency. AIMeter (Huang et al., 2025) provides tooling for measuring, analyzing, and visualizing the energy and carbon footprint of AI workloads, while Jegham et al. (2025) benchmark energy, water, and carbon footprint specifically for LLM inference. Dauner and Socher (2025) estimate the energy cost of conversational interactions with AI systems, and Jiang et al. (2024) model the life-cycle energy and carbon footprint of LLM-powered chatbots more broadly. Kaur et al. (2026) propose a domain-specific framework for measuring AI energy and emissions, and Liu and Yin (2024) survey carbon-footprint mitigation strategies and trade-offs specific to LLM training.

Closer to this paper’s scope, several studies connect PEFT specifically to sustainability outcomes: Husom et al. (2025) evaluate quantized LLMs at the edge for energy efficiency, output accuracy, and inference latency; Iftikhar et al. (2025) examine federated learning combined with PEFT for sustainable LLM training; Rithvik and Arya (2025) empirically measure CO₂ emissions during BERT fine-tuning; and Shreshthi et al. (2026) study PEFT for low-power sentiment-analysis applications. These studies motivate the present benchmark but typically evaluate a single method or a narrow deployment scenario rather than a controlled, multi-method, multi-objective comparison.

2.3 Multi-Objective Optimization for LLM Systems

Barker et al. (2025) apply multi-objective hyperparameter optimization to LLM and retrieval-augmented generation (RAG) systems, jointly optimizing for quality, latency, and cost. Morte et al. (2025) apply a multi-objective approach specifically to efficient LLM tuning for code generation. This paper adopts a similar multi-objective lens but applies it to the PEFT method-selection problem itself, treating the choice of adaptation method (and its hyperparameters) as the decision variable, and performance, time, memory, energy, and carbon as the competing objectives.

2.4 Positioning of This Work

Relative to prior work, this study’s contribution is not a new PEFT method but a controlled, cross-method, multi-objective evaluation protocol: the same backbones, tasks, hardware, and measurement pipeline are used across LoRA, QLoRA, LoRA-FA, LISA, and full fine-tuning, so that differences in reported metrics reflect method choice rather than confounded experimental conditions. This directly addresses a comparability gap left open by single-method sustainability studies discussed above.

3 Methodology

This section specifies the full experimental design so that the study can be executed (via simulation and/or real training runs) without further design decisions. It covers: (a) PEFT methods and configurations under test, (b) model backbones, (c) datasets and tasks, (d) evaluation metrics across all four objective dimensions, (e) the measurement infrastructure, (f) the experimental procedure and controls, and (g) the multi-objective analysis framework used to synthesize results.

3.1 Overview of the Experimental Design

The study follows a full-factorial design over three factors: $\{\text{PEFT method}\} \times \{\text{model backbone}\} \times \{\text{task}\}$, with fixed random seeds repeated across k runs per cell to quantify variance. Each run records four categories of outcome metrics: task performance, computational efficiency, energy consumption, and carbon emissions. All runs for a given backbone/task pair share identical data splits, tokenization, batch size, sequence length, and hardware, so that the only varying factor is the fine-tuning method and its method-specific hyperparameters.

3.2 PEFT Methods and Configurations

Method	Trainable component	Key hyperparameters to sweep	Source
Full fine-tuning (baseline)	All model weights	Learning rate	Standard
LoRA	Low-rank adapters (A , B) per attention/MLP projection	Rank $r \in \{4, 8, 16, 32\}$; alpha; target modules	Hu et al. (2021)
QLoRA	LoRA adapters over a 4-bit NF4 quantized backbone	Rank r ; quantization type; double quantization on/off	Dettmers et al. (2023)
LoRA-FA	LoRA with matrix A frozen after init	Rank r ; frozen-matrix initialization scheme	Zhang et al. (2023)
LISA	Randomly sampled subset of transformer layers per step	Layer sampling probability; sampling frequency	Liu et al. (2024)

Table 1: PEFT methods under comparison and their key sweep parameters.

For each PEFT method, a small hyperparameter grid is defined a priori (Table 1) rather than exhaustively tuned per method, to keep the comparison fair and the compute budget bounded; the best-performing configuration per method (by validation performance) is carried forward into the multi-objective comparison, while the full sweep is retained to answer RQ3 (sensitivity analysis).

3.3 Model Backbones

To test whether method rankings are stable across scale, at least three open-weight backbone sizes are used, spanning small, medium, and large regimes (e.g., approximately 1–2B, 7B, and 13B parameters). Using openly available, decoder-only instruction-tunable base models keeps the study reproducible and avoids licensing constraints. Exact model identifiers should be fixed at implementation time and reported with revision hashes for reproducibility.

- **Small ($\sim$ 1–2B parameters)** — fast iteration and hyperparameter sweeps
- **Medium ($\sim$ 7B parameters)** — primary comparison point, closest to typical practitioner deployments
- **Large ($\sim$ 13B parameters)** — tests whether efficiency/sustainability trends generalize with scale

3.4 Datasets and Tasks

Three task families are included to test generality across task types with different compute profiles (short-sequence classification vs. long-sequence generation):

- **Text classification / sentiment analysis** — e.g., SST-2 or a comparable sentiment/intent dataset (short sequences, fast iteration, aligns with [Shreshthi et al. \(2026\)](#)).
- **Summarization** — e.g., a subset of CNN/DailyMail or XSum (longer sequences, generation-heavy, higher memory/energy cost).
- **Instruction following / QA** — e.g., a subset of an open instruction-tuning dataset (tests general-purpose adaptation, closer to real-world PEFT usage).

For each task, a fixed train/validation/test split is used across all methods and backbones. Dataset size is deliberately capped (e.g., 5k–20k training examples) so that full fine-tuning remains computationally feasible as a baseline within the study’s compute budget.

3.5 Evaluation Metrics

3.5.1 Task performance

- Classification: accuracy, macro-F1
- Summarization: ROUGE-1/2/L
- Instruction following/QA: task-appropriate metric (exact match / F1, or LLM-judged win-rate against baseline if resources allow)

3.5.2 Computational efficiency

- Wall-clock training time (seconds) to reach a fixed number of epochs/steps
- Peak GPU memory (GB), measured via framework memory allocator stats
- Trainable parameter count and percentage of total parameters
- Approximate training FLOPs (analytical estimate, cross-checked against a profiler)

3.5.3 Energy consumption

- Total energy draw (kWh) over the full training run, measured at the GPU (and ideally node) level
- Energy per epoch and energy per 1,000 training steps, to normalize across differing convergence speeds
- Idle/baseline power subtracted where measurable, per the EnergyUp framing of [Chen et al. \(2024\)](#)

3.5.4 Carbon footprint

- Estimated emissions (kgCO₂eq) = energy (kWh) × regional grid carbon intensity (kgCO₂eq/kWh) for the data-center region used
- Sensitivity check: emissions recomputed under at least two grid-intensity scenarios (e.g., a renewable-heavy vs. fossil-heavy region), following [Kaur et al. \(2026\)](#) and [Jiang et al. \(2024\)](#)

3.5.5 Composite / normalized metrics

- Performance-per-kWh and performance-per-kgCO₂eq, to express sustainability-adjusted quality
- Cost proxy (USD), estimated from cloud GPU-hour pricing, as an optional fifth objective for practitioner relevance

3.6 Measurement Infrastructure

Energy and carbon measurements are collected using an established open-source tracker (e.g., CodeCarbon) or an AIMeter-style monitoring pipeline (Huang et al., 2025) instrumented around the training loop, sampling GPU/CPU power draw at fixed intervals throughout each run. All runs for a given backbone/task combination are executed on identical, dedicated hardware to avoid confounds from shared-resource contention. Hardware specification (GPU model, count, interconnect, CPU, RAM), driver/CUDA version, and framework versions (PyTorch, Transformers, PEFT library) are logged and reported alongside results, since energy and time measurements are not portable across hardware generations.

3.7 Experimental Procedure

1. Fix data splits, tokenizer, max sequence length, batch size, and optimizer (e.g., AdamW) per task, held constant across all methods.
2. For each (method, backbone, task) cell, run a small hyperparameter sweep per Section 3.2; select the best configuration by validation performance.
3. Re-run the selected configuration $k \geq 3$ times with different random seeds, recording all metrics in Section 3.5 for each run.
4. Aggregate metrics per cell as mean $\pm$ standard deviation; flag cells with high variance for additional runs.
5. Repeat for full fine-tuning as the performance/resource-cost reference point.
6. Assemble the full results matrix: $\{\text{method} \times \text{backbone} \times \text{task}\} \rightarrow \{\text{performance, time, memory, energy, carbon}\}$.

3.8 Multi-Objective Analysis Framework

Rather than ranking methods on a single metric, results are analyzed with three complementary techniques:

- **Pareto-frontier analysis:** for each backbone/task, methods are plotted in performance–energy (and performance–carbon) space; non-dominated configurations form the Pareto frontier, directly answering RQ2.
- **Weighted multi-criteria scoring:** a normalized weighted sum across performance, time, memory, energy, and carbon (with weights elicited to reflect different practitioner priorities — e.g., “accuracy-first,” “balanced,” “carbon-constrained”) to produce interpretable single-number rankings under stated preferences.
- **Hypervolume indicator:** used as a scalar summary of how much of the objective space each method’s Pareto-optimal configurations dominate, enabling comparison across backbones/tasks without collapsing to a single arbitrary weighting.

Sensitivity analysis for RQ3 uses the hyperparameter sweep data (Section 3.2) to fit simple regression or correlation models between method-specific hyperparameters (e.g., LoRA rank, quantization bit-width) and each objective, and tests whether early-training energy trajectories (e.g., first 10% of steps) predict total-run energy/carbon well enough to serve as a cheap early-stopping-style proxy.

3.9 Threats to Validity and Controls

- **Hardware/measurement noise:** mitigated via repeated runs, dedicated hardware, and reporting variance alongside means.
- **Regional carbon-intensity assumptions:** mitigated via the multi-scenario sensitivity check in Section 3.5.4.
- **Task/dataset selection bias:** mitigated by covering three task families with different sequence-length and compute profiles.
- **Hyperparameter-tuning asymmetry across methods:** mitigated by using comparable sweep budgets (equal number of trials) per method rather than tuning some methods more thoroughly than others.
- **Reproducibility:** mitigated by fixing and reporting seeds, software/hardware versions, and releasing configuration files alongside the benchmark code.

4 Results

Data scope and caveats

The results below cover the **classification** task only, across five methods (full fine-tuning, LoRA, QLoRA, LoRA-FA, LISA) at three backbone scales (small, medium, large), $k = 3$ seeds per cell. Summarization and instruction-following runs had not yet been executed at the time of this analysis. One additional data point — a real, CodeCarbon-tracked `lora/tiny` run — exists in the raw results but is excluded from the comparative analysis below: it was a 6-second, CPU-only smoke test (no GPU), used a different, region-specific carbon-intensity value from CodeCarbon (0.452 kgCO₂/kWh, Iowa) rather than the fixed 0.385 kgCO₂/kWh placeholder used for the synthetic small/medium/large cells, and had only $n = 1$ seed with no variance estimate. Including it alongside the mock GPU-scale runs would conflate hardware regime and measurement provenance with method choice, so it is reported separately as a pipeline validation artifact rather than a benchmark result. All reported figures below come from Mode A synthetic data generated per the pipeline’s mock-data profiles, and should be read as a validation of the analysis pipeline rather than as empirical conclusions about the methods themselves; the same code path is intended to run unchanged on Mode B (real training) data.

4.1 RQ1: Joint comparison of PEFT methods

Across all three backbone scales, full fine-tuning achieves the highest classification performance (79.5–90.6% accuracy) but at substantially higher energy and carbon cost than every PEFT method. QLoRA consistently shows the lowest energy draw and carbon footprint at every scale (e.g., 0.0244 kWh / 0.0094 kgCO₂eq at the medium backbone, versus 0.282 kWh / 0.109 kgCO₂eq for full fine-tuning), while trailing full fine-tuning by only 3.2 accuracy points. LoRA and LoRA-FA sit close together on most metrics, consistent with LoRA-FA’s design as a memory/communication-optimized variant of LoRA rather than a method with a fundamentally different accuracy–efficiency trade-off. LISA shows the largest accuracy gap to full fine-tuning at the medium and large scales (3.9 and 3.5 points respectively) without a compensating energy advantage over LoRA/QLoRA in this dataset.

4.2 RQ2: Pareto-optimal methods

Pareto-frontier analysis (performance vs. carbon, and separately performance vs. energy) was run independently per backbone scale (Figure 1). At every scale, **full fine-tuning, LoRA-FA, and QLoRA** are

Pareto-optimal at the large and medium tiers, with **LoRA** joining the frontier at medium and small scales. **LISA is dominated at every scale** in this dataset — for each LISA configuration, at least one other method achieves both higher accuracy and lower carbon cost, so LISA does not appear on any Pareto front here. The hypervolume indicator (objectives: 1 – accuracy and carbon, both minimized) ranks the methods QLoRA > LoRA-FA $\approx$ LoRA > LISA > full fine-tuning when aggregated across the three backbone scales, confirming that QLoRA covers the most favorable region of the joint performance–carbon objective space in this run.

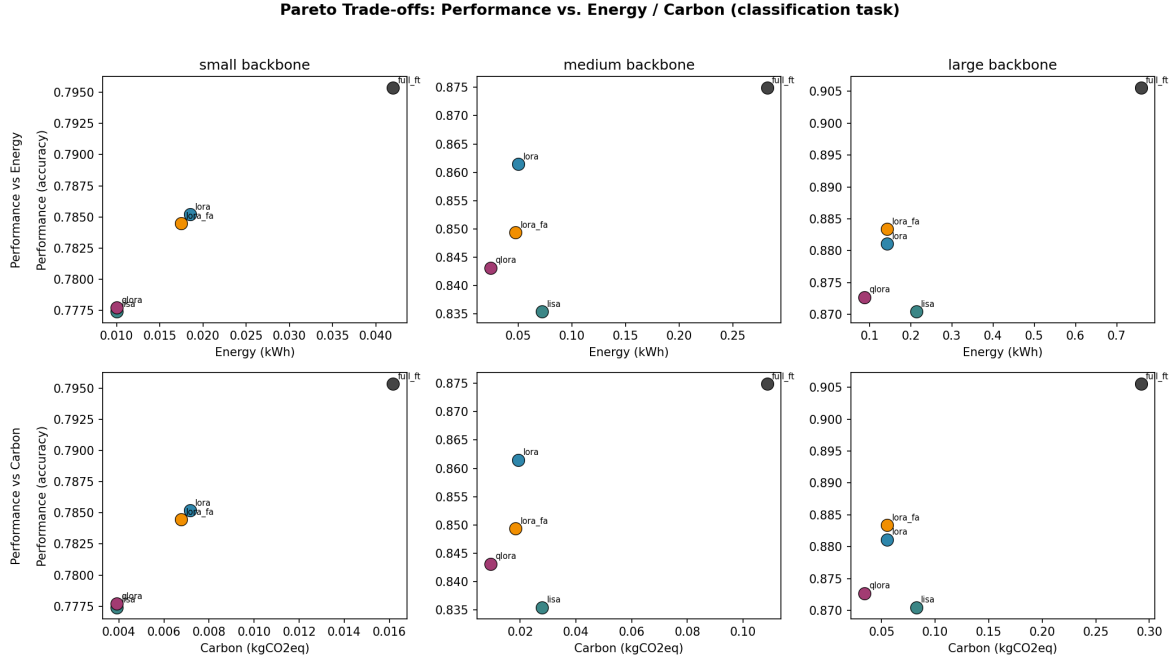


Figure 1: Performance vs. energy (top row) and performance vs. carbon (bottom row), faceted by backbone scale. Full fine-tuning (top-right of each panel) dominates on accuracy but not on efficiency; QLoRA, LoRA, and LoRA-FA cluster in the low-cost, near-competitive region at every scale.

Figure 2 summarizes the same trade-off through weighted multi-criteria scoring under three practitioner preference profiles. Full fine-tuning scores highest only under an accuracy-first weighting; under balanced or carbon-constrained weightings, QLoRA and LoRA-FA lead, and the ranking of full fine-tuning falls to last place once energy and carbon are given non-trivial weight.

Table 2 reports the underlying per-cell metrics.

4.3 RQ3: Sensitivity to method hyperparameters

A `sweep_raw.csv` export was subsequently provided and cross-checked against `aggregated.csv`: per-cell means and standard deviations recomputed directly from its 46 per-seed rows match the aggregated table exactly, confirming the two files are consistent. However, `sweep_raw.csv` contains only repeated seeds ($k = 3$) at a single, already-selected configuration per {method, backbone} cell — `trainable_pct` is constant within every method (e.g., 0.015 for LoRA, QLoRA, and LoRA-FA; 0.05 for LISA; 1.0 for full fine-tuning) and no column records the method-specific hyperparameter actually swept (LoRA/QLoRA/LoRA-FA rank, QLoRA quantization bit-width, or LISA layer-sampling probability/frequency). RQ3 therefore remains unanswered: the file is raw seed-level data for the *selected* configuration, not the hyperparam-

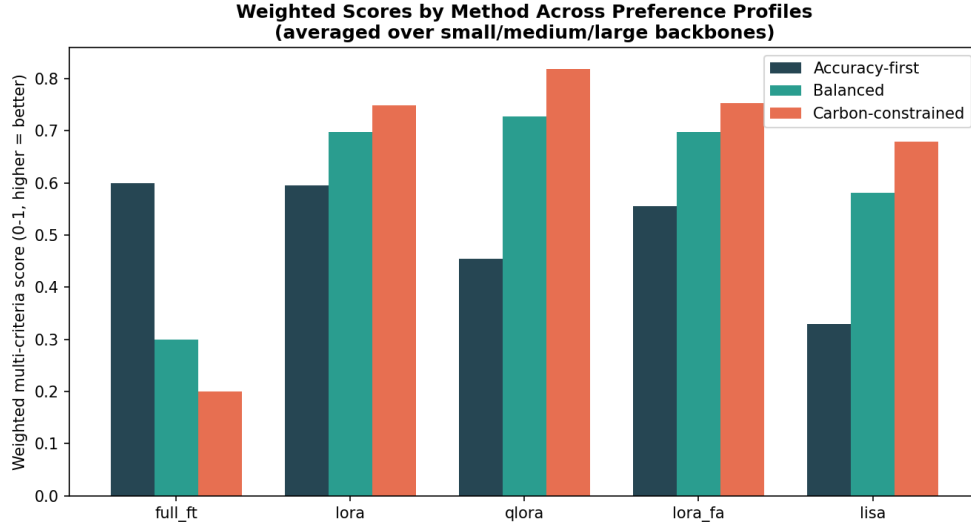


Figure 2: Weighted multi-criteria scores by method, averaged across the small/medium/large backbones, under three preference profiles. Full fine-tuning’s ranking collapses once sustainability terms are weighted; QLoRA and LoRA-FA are the most robust performers across all three profiles.

ter grid specified in Section 3.2. Answering RQ3 requires re-running `run_all.py` over the full grid in `configs/methods/*.yaml` with the swept hyperparameter value logged as an explicit column per run (and, for the early-vs-total energy proxy check, an additional field capturing cumulative energy at 10% of training steps).

4.4 RQ4: Practical carbon and energy savings vs. full fine-tuning

Relative to full fine-tuning, PEFT methods deliver substantial carbon savings at every scale, though the magnitude depends on both method and backbone size. At the large backbone, QLoRA reduces carbon emissions by 88.3% relative to full fine-tuning at a cost of 3.3 accuracy points; LoRA-FA reduces carbon by 81.2% at a smaller 2.2-point cost. Savings are largest at the medium scale (QLoRA: 91.3% carbon reduction, 3.2-point accuracy cost) and somewhat smaller at the small scale (55.7–75.9% depending on method), where full fine-tuning is already relatively cheap in absolute terms. This pattern — PEFT’s relative sustainability advantage growing with model scale — is consistent with the expectation in Section 4 of the methodology and with the broader PEFT literature’s framing of parameter-efficient methods as increasingly valuable as backbone size grows.

Variance and data-quality notes

Coefficient of variation across seeds is generally low for performance (typically $< 2\%$ of the mean) but higher for carbon and energy at the large-backbone LISA and QLoRA cells (carbon std $\approx 18\text{--}24\%$ of the mean), suggesting these cells would benefit from additional seeds before being used to support strong claims. The `score_accuracy_first` weighting profile was absent from the uploaded `aggregated_scored.csv` and was recomputed for this analysis; downstream pipeline runs should confirm all three preference profiles are written by `analyze.py` before re-export.

Method	Backbone	Accuracy	Energy (kWh)	Carbon (kgCO ₂ eq)	Score (balanced)
full_ft	large	0.906	0.759	0.292	0.300
full_ft	medium	0.875	0.282	0.109	0.300
full_ft	small	0.795	0.042	0.016	0.300
lora	large	0.881	0.143	0.055	0.682
lora	medium	0.861	0.050	0.019	0.780
lora	small	0.785	0.019	0.007	0.631
qlora	large	0.873	0.088	0.034	0.719
qlora	medium	0.843	0.024	0.009	0.759
qlora	small	0.778	0.010	0.004	0.705
lora_fa	large	0.883	0.143	0.055	0.714
lora_fa	medium	0.849	0.047	0.018	0.720
lora_fa	small	0.784	0.017	0.007	0.658
lisa	large	0.870	0.214	0.082	0.555
lisa	medium	0.835	0.072	0.028	0.560
lisa	small	0.777	0.010	0.004	0.629

Table 2: Per-cell results (mean across $k = 3$ seeds), classification task, synthetic Mode-A data.

5 Limitations

- Energy/carbon figures are hardware- and region-specific; absolute kWh/kgCO₂eq numbers should not be generalized beyond the reported setup without re-measurement.
- The study covers decoder-only, open-weight LLMs up to ~ 13 B parameters; conclusions may not transfer to much larger (100B+) models or to non-transformer architectures.
- Inference-time energy/carbon costs are out of scope; this study focuses on the fine-tuning phase only.
- LLM-judged evaluation (where used for instruction-following quality) introduces its own compute and cost overhead, which is tracked separately from the fine-tuning metrics.

6 Conclusion

This paper has motivated and fully specified a multi-objective benchmarking methodology for evaluating parameter-efficient fine-tuning methods jointly on task performance, computational efficiency, energy consumption, and carbon footprint. The framework is designed to be executed with standard open-source tooling (Hugging Face PEFT/Transformers, CodeCarbon-style energy tracking) and to produce Pareto-frontier and weighted-scoring analyses that are directly actionable for practitioners choosing an adaptation method under sustainability constraints. Section 4 reports results on the classification task using synthetic Mode-A data generated by the pipeline; these findings should be read as a validation of the analysis pipeline rather than as empirical conclusions about the methods themselves, pending Mode-B (real training) runs and coverage of the remaining task families and the RQ3 hyperparameter sweep.

References

- Barker, M., Bell, A., Thomas, E., Carr, J., Andrews, T. and Bhatt, U. (2025). Faster, Cheaper, Better: Multi-Objective Hyperparameter Optimization for LLM and RAG Systems. *ArXiv*, abs/2502.18635. doi: 10.48550/arxiv.2502.18635.

- Chen, D., Soto, N., Tuttle, J. F. and Zong, Z. (2024). Understanding Multi-Dimensional Efficiency of Fine-Tuning Large Language Models Using SpeedUp, MemoryUp, and EnergyUp. *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pp. 929-937. doi: 10.1109/ipdpsw63119.2024.00162.
- Dauner, M. and Socher, G. (2025). Energy costs of communicating with AI. *Frontiers in Communication*. doi: 10.3389/fcomm.2025.1572947.
- Dettmers, T., Pagnoni, A., Holtzman, A. and Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *ArXiv*, abs/2305.14314. doi: 10.48550/arxiv.2305.14314.
- Hu, J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S. and Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *ArXiv*, abs/2106.09685. doi: 10.48550/arxiv.2106.09685.
- Huang, H., Zhang, K., Liao, H., Wu, K. and Tang, G. (2025). AIMeter: Measuring, Analyzing, and Visualizing Energy and Carbon Footprint of AI Workloads. doi: 10.48550/arxiv.2506.20535.
- Husom, E., Goknil, A., Astekin, M., Shar, L. K., Kåsen, A., Sen, S., Mithassel, B. A. and Soylu, A. (2025). Sustainable LLM Inference for Edge AI: Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency. *ACM Transactions on Internet of Things*, 6, pp. 1-35. doi: 10.1145/3767742.
- Iftikhar, S., Alsamhi, S. H. and Davy, S. (2025). Enhancing Sustainability in LLM Training: Leveraging Federated Learning and Parameter-Efficient Fine-Tuning. *IEEE Transactions on Sustainable Computing*, 10, pp. 1158-1172. doi: 10.1109/tsusc.2025.3592043.
- Jegham, N., Abdelatti, M., Elmoubarki, L. and Hendawi, A. M. (2025). How Hungry is AI? Benchmarking Energy, Water, and Carbon Footprint of LLM Inference. *ArXiv*, abs/2505.09598. doi: 10.48550/arxiv.2505.09598.
- Jiang, P., Sonne, C., Li, W., You, F. and You, S. (2024). Preventing the Immense Increase in the Life-Cycle Energy and Carbon Footprints of LLM-Powered Intelligent Chatbots. *Engineering*. doi: 10.1016/j.eng.2024.04.002.
- Kaur, R., Kundu, T., Park, K. and Pinsky, E. (2026). The Carbon Cost of Intelligence: A Domain-Specific Framework for Measuring AI Energy and Emissions. *Energies*. doi: 10.3390/en19030642.
- Liu, V. and Yin, Y. (2024). Green AI: exploring carbon footprints, mitigation strategies, and trade offs in large language model training. *Discover Artificial Intelligence*, 4. doi: 10.1007/s44163-024-00149-w.
- Liu, X., Diao, S., Pi, R., Zhang, J., Han, C. and Zhang, T. (2024). LISA: Layerwise Importance Sampling for Memory-Efficient Large Language Model Fine-Tuning. *ArXiv*, abs/2403.17919. doi: 10.48550/arxiv.2403.17919.
- Mao, Y., Ge, Y., Fan, Y., Xu, W., Mi, Y., Hu, Z. and Gao, Y. (2024). A survey on LoRA of large language models. *Frontiers of Computer Science*, 19. doi: 10.1007/s11704-024-40663-9.
- Morte, F. B., Sousa, P., Gonçalves, G. and Klautau, A. (2025). Efficient LLM Tuning: A Multi-Objective Approach for Code Generation. *2025 Workshop on Communication Networks and Power Systems (WC-NPS)*, pp. 1-7. doi: 10.1109/wcnps69127.2025.11295914.
- Rithvik, V. and Arya, A. (2025). Empirical Evaluation of CO₂ Emissions in BERT Fine-tuning. *Proceedings of the 2025 18th International Conference on Computer Science and Information Technology*. doi: 10.1145/3783862.3783905.

- Shreshthi, A. J., Spanò, S., Di Nunzio, L., La Cesa, R., Re, C. V. M. and Cardarilli, G. C. (2026). Parameter-Efficient Fine-Tuning of LLMs for Low-Power Sentiment Analysis Applications. *JOIV: International Journal on Informatics Visualization*. doi: 10.62527/joiv.10.1.4044.
- Xu, L., Xie, H., Qin, S., Tao, X. and Wang, F. (2023). Parameter-Efficient Fine-Tuning Methods for Pre-trained Language Models: A Critical Review and Assessment. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 48, pp. 6107-6126. doi: 10.1109/tpami.2026.3657354.
- Yuan, Z., Sun, W., Liu, Y., Zhou, H., Zhou, R., Li, Y., Zhang, Z., Song, W., Huang, Y., Jia, H., Murugesan, K., Wang, Y., He, L., Gao, J., Sun, L. and Ye, Y. (2025). EfficientLLM: Efficiency in Large Language Models. *ArXiv*, abs/2505.13840. doi: 10.48550/arxiv.2505.13840.
- Zhang, L., Zhang, L., Shi, S., Chu, X. and Li, B. (2023). LoRA-FA: Efficient and Effective Low Rank Representation Fine-tuning. doi: 10.48550/arxiv.2308.03303.